

Artemis Bot

Relatorio executivo, auditoria comparativa e proposta de conclusao

Assistente de IA para WhatsApp com integrações de CRM, agenda e pagamento

Cliente: Confluence Treinamento

Base analisada: Repositorio local Artemis-Bot + planejamento em PDF

Data: 24/04/2026

Status: Desenvolvimento avancado, pendente de endurecimento tecnico

Sumário

1 Mensagem executiva ao cliente	2
2 Resumo de valor entregue	2
2.1 O que o cliente recebe	2
2.2 Por que isso tem valor comercial	2
3 Estimativa comercial de conclusao	3
3.1 Premissas	3
3.2 Escopo recomendado para fechar o projeto	3
3.3 Proposta de investimento	3
3.4 Preço recomendado para esta oportunidade	4
3.5 Como defender o preço	4
4 Veredito tecnico resumido	4
4.1 Pontos fortes	4
4.2 Riscos antes de producao	5
5 Inventario real do projeto	5
5.1 Arquivos encontrados	5
5.2 Contagem local	6
6 Matriz comparativa	6
7 Comparativo por area funcional	7
7.1 Arquitetura principal	7
7.2 Inteligencia artificial	7
7.3 Funil comercial	7
7.4 Asaas	8
7.5 Google Calendar	8
7.6 ResponDi	8
7.7 LGPD	8
8 O que ja foi feito	8
9 O que falta fazer	9
9.1 Prioridade critica	9
9.2 Prioridade alta	9
9.3 Prioridade media	9
10 Plano de entrega recomendado	10
11 Criterios de aceite	10
12 Conclusao	10

1 Mensagem executiva ao cliente

O Artemis Bot ja ultrapassou a fase de ideia. O que existe hoje nao e apenas um chatbot simples: e uma aplicacao TypeScript sob medida, integrada ao WhatsApp oficial, banco de dados, IA generativa, Google Calendar, Asaas e formulario ResponDi. Isso significa que o projeto ja tem uma base real de produto, com automacao de atendimento, qualificacao comercial, agendamento e cobranca.

O ponto central da avaliacao e este: o projeto foi iniciado com apoio de IA e ja possui uma quantidade relevante de implementacao, mas ainda precisa passar por uma etapa profissional de conclusao. Essa etapa nao e simplesmente “continuar escrevendo codigo”. O trabalho correto agora e transformar uma base funcional em uma entrega confiavel, segura, documentada e defensavel perante o cliente final.

Como CEO ou responsavel pela entrega, eu apresentaria o Artemis da seguinte forma:

O projeto Artemis ja tem o nucleo do produto implementado. Ele conversa pelo WhatsApp, entende o estagio do lead, salva historico, qualifica o aluno, apresenta programas, trata objecoes, agenda no calendario e gera cobranças. A etapa restante e a mais importante para uma entrega profissional: segurança, testes, validacao de fluxos, estabilizacao de integracoes, documentacao operacional e preparacao para producao.

O valor agregado nao esta apenas no bot responder mensagens. O valor esta em reduzir trabalho manual, responder leads com consistencia, preservar contexto, encaminhar o cliente pelo funil comercial e automatizar pontos que normalmente dependem de uma pessoa: cadastro, agenda, pagamento e confirmacao.

2 Resumo de valor entregue

2.1 O que o cliente recebe

- Um assistente de WhatsApp com IA personalizado para a Confluence Treinamento.
- Uma arquitetura propria em Node.js e TypeScript, sem depender de construtores visuais limitados.
- Memoria persistente de conversas em PostgreSQL.
- Fluxo comercial por estados: saudacao, qualificacao, apresentacao, objecao, fechamento e humano.
- Integracao com Meta WhatsApp Business API.
- Integracao com Google Gemini para respostas e extracao de dados.
- Integracao com Google Calendar para agenda.
- Integracao com Asaas para cliente, cobranca, assinatura e cancelamento.
- Integracao com ResponDi para formulario de matricula.
- Regras de LGPD: consentimento, exclusao de dados e retencao de leads inativos.
- Configuracao de persona, programas e mensagens sem alterar codigo.

2.2 Por que isso tem valor comercial

Um atendimento humano pode perder leads por demora, inconsistência ou falta de follow-up. O Artemis tenta resolver esse gargalo com disponibilidade continua e processo padronizado. Alem disso, ao integrar agenda e pagamento, o sistema reduz a distancia entre “tenho interesse” e “matricula iniciada”.

Na pratica, o ativo entregue e uma automacao comercial com IA, nao apenas uma conversa automatica. Isso justifica tratar o projeto como software sob medida, com fase de estabilizacao e manutencao, e nao como configuracao simples de chatbot.

3 Estimativa comercial de conclusao

3.1 Premissas

A estimativa abaixo considera que o cliente ja trouxe uma base iniciada e que o trabalho contratado agora seria concluir, estabilizar e preparar a entrega para uso real. Nao inclui custos de terceiros, como Meta, Asaas, Google, VPS, dominio, certificados, banco gerenciado ou mensalidade de APIs.

Referencias publicas recentes de mercado indicam ampla variacao de custo para desenvolvimento sob medida e horas tecnicas senior no Brasil. Foram usadas apenas como baliza externa, nao como tabela obrigatoria: ha fontes citando hora de desenvolvimento na faixa de R\$ 120–180/h para software sob medida, faixa ampla de freelancer autonomo entre R\$ 68–243/h e projetos de software variando muito conforme risco e complexidade.¹²³

3.2 Escopo recomendado para fechar o projeto

Frente	Esforço mado	esti-	Entrega
Auditoria tecnica e organizacao	12–20 h		Revisao da base, definicao de riscos, ajuste de README, envs e runbook.
Seguranca e producao	24–40 h		Validacao webhook Asaas, remocao de logs sensiveis, graceful shutdown, Calendar ID por ambiente.
Banco e infraestrutura de app	16–28 h		Prisma Singleton, migrations, historico correto, ajustes de deploy.
Testes e QA	32–56 h		Testes unitarios de FSM, normalizador, pagamentos, prompts e checklist end-to-end.
Refinamento de funil	24–44 h		Ajuste de protocolo teen, PRM, midias nao suportadas, fechamento e handoff.
Deploy assistido e transferencia	16–28 h		Subida controlada, validacao sandbox, documentacao operacional e treinamento.

3.3 Proposta de investimento

Para uma entrega profissional, eu recomendaria apresentar tres opcoes:

Plano	Prazo	Investimento	Quando usar
Essencial	2–3 semanas	R\$ 18.000–26.000	Corrigir riscos criticos, documentar e deixar rodando com QA manual limitado.

¹<https://www.aegisai.com.br/blog/preco-desenvolvimento-software-sob-medida-2026>

²<https://www.freelasemcrise.com.br/quanto-cobra/desenvolvedor-software/sao-paulo>

³<https://www.neryx.com.br/blog/custo-desenvolvimento-software-brasil-2026/>

Profissional	4–6 semanas	R\$ 32.000–48.000	Recomendado. Inclui seguranca, testes, QA, deploy assistido e documentacao de operacao.
Produto	8–10 semanas	R\$ 58.000–85.000	Inclui melhorias de produto, painel/observabilidade inicial, status de matricula e preparacao para evolucao SaaS.

3.4 Preco recomendado para esta oportunidade

Para este caso especifico, considerando que ja existe uma base funcional e que o cliente quer concluir com seguranca, minha recomendacao comercial seria:

Proposta recomendada: R\$ 38.000

Prazo estimado: 5 semanas

Escopo: estabilizacao tecnica, seguranca, testes essenciais, QA completo do funil, documentacao e deploy assistido.

Manutencao opcional apos entrega:

- R\$ 2.500–3.500/mês para suporte leve, ajustes de prompt e acompanhamento.
- R\$ 4.500–6.500/mês para suporte com evolucoes mensais, monitoramento e melhorias continuas.

3.5 Como defender o preco

O argumento nao deve ser “vou terminar um bot”. O argumento correto e:

Vamos transformar um prototipo avancado em um sistema operacional de vendas e atendimento, com integracoes reais de calendario, pagamento, dados e WhatsApp oficial. O trabalho inclui seguranca, testes, estabilidade e transferencia para que a Confluence consiga usar isso como ativo de negocio, nao como experimento.

4 Veredito tecnico resumido

O projeto real esta muito proximo do planejamento em arquitetura geral, stack, integracoes principais e intencao de produto. A base tem os 16 arquivos TypeScript citados no planejamento e soma 2.728 linhas de TypeScript em `src/`, alinhado com o documento original.

4.1 Pontos fortes

- Estrutura modular clara: controllers, services, flow, types e utils.
- FSM de conversa implementada.
- Debounce por usuario no webhook principal.
- Prompt dinamico por estado em XML.
- Gemini com tool calling para agenda e pagamentos.
- Asaas para cliente, cobranca, assinatura e cancelamento.
- Google Calendar para disponibilidade, criacao, busca e cancelamento.

- Respondi com validacao por token e normalizacao de telefone.
- Banco PostgreSQL via Prisma.
- Preocupacao real com LGPD.

4.2 Riscos antes de producao

- Webhook Asaas aceita eventos sem assinatura, secret ou verificacao de origem.
- Existem multiplas instancias diretas de `PrismaClient`.
- Nao existe suite de testes.
- Nao existem migrations Prisma versionadas.
- Nao existe `.env.example`, configuracao PM2 ou runbook de deploy completo no repositorio.
- README esta desatualizado e cita OpenAI/variaveis antigas.
- Calendar ID esta hardcoded para um email pessoal/teste.
- Nao ha graceful shutdown.
- Parte do comportamento critico depende de prompt, nao de validacao backend.

5 Inventario real do projeto

5.1 Arquivos encontrados

Arquivo	Funcao no projeto
<code>package.json</code>	Define scripts, dependencias e entrada de producao. Contem Express, Prisma, Gemini, Google APIs, Axios e utilitarios.
<code>package-lock.json</code>	Trava versoes instaladas das dependencias npm.
<code>tsconfig.json</code>	Configura compilacao TypeScript para CommonJS, target ES2020 e saida em <code>dist/</code> .
<code>prisma.config.ts</code>	Configuracao Prisma apontando para <code>prisma/schema.prisma</code> e <code>DATABASE_URL</code> .
<code>README.md</code>	Documentacao inicial, porem desatualizada em variaveis e provedor de IA.
<code>prisma/schema.prisma</code>	Define modelos <code>User</code> e <code>ChatHistory</code> .
<code>config/persona.json</code>	Persona, tom, restricoes, objecoos, contratos e links de protocolo.
<code>config/programs.json</code>	Programas vendidos, precos, duracao e textos informativos.
<code>config/settings.json</code>	Mensagens e numeros de suporte/handoff.
<code>src/index.ts</code>	Inicializa Express, registra rotas e cron LGPD.
<code>src/controllers/WebhookController.ts</code>	Webhook principal WhatsApp, debounce, processamento background, handoff e envio de respostas.

src/controllers/AsaasWebhookController.ts	Recebe eventos Asaas e envia confirmacao de pagamento ao usuario.
src/controllers/RespondiController.ts	Recebe formulario Respondi, valida token, extrai campos e faz upsert do usuario.
src/flow/StateResolver.ts	Resolve estado conversacional e tipos de transferencia humana.
src/services/AIService.ts	Integra Gemini, extrai perfil, gera resposta e executa ferramentas de agenda/pagamento.
src/services/AsaasService.ts	Busca/cria clientes, gera cobranças/assinaturas e cancela pendências no Asaas.
src/services/CalendarService.ts	Verifica disponibilidade, cria, busca e cancela eventos no Google Calendar.
src/services/ConfigLoader.ts	Carrega arquivos JSON de configuracao.
src/services/PromptBuilder.ts	Monta prompt XML por estado da conversa.
src/services/StateService.ts	Gerencia usuario, historico, perfil, estado e exclusao LGPD.
src/services/WhatsAppService.ts	Envia mensagens pela Meta Graph API.
src/types/user.ts	Define estados de conversa e tipos de usuario.
src/types/config.ts	Define tipos TypeScript para persona, programas e settings.
src/utils/prisma.ts	Singleton Prisma para acesso ao banco.
src/utils/phoneNormalizer.ts	Normaliza telefones brasileiros para formato canonico.

5.2 Contagem local

Grupo	Linhas
TypeScript em src/	2.728
Config JSON	119
Prisma schema	50
README	51
Total auditado localmente	3.059

6 Matriz comparativa

Area	Status	Leitura tecnica
TypeScript/Node.js	Implementado	Stack real bate com o planejamento.
Express HTTP API	Implementado	Rotas principais existem.
PostgreSQL + Prisma	Parcial	Schema existe, mas sem migrations versionadas.
User + ChatHistory	Implementado	Campos batem com o planejamento.
FSM com 6 estados	Implementado	Estado e resolucao implementados em TypeScript.
Debounce 5s	Implementado	Buffer por usuario existe.

Resposta 200 imediata Meta	Implementado	ACK acontece antes do processamento pesado.
PromptBuilder XML	Implementado	Ponto forte da arquitetura.
Gemini principal	Implementado	Usa <code>gemini-3.1-pro-preview</code> .
Gemini extracao JSON	Implementado	Usa <code>gemini-2.5-flash</code> .
Tool calling 6 ferramentas	Implementado	4 ferramentas Calendar + 2 Asaas.
Filtro de raciocinio	Parcial	Existe heuristica, mas nao e garantia formal.
Google Calendar	Parcial	Funcional, mas Calendar ID hardcoded.
Asaas	Parcial	Cobranca existe; webhook sem validacao e risco critico.
Respondi	Implementado	Valida token e faz upsert.
LGPD exclusao	Implementado	Delecao com cascade implementada.
LGPD retencao 30 dias	Parcial	Cron existe, mas usa Prisma separado e <code>setInterval</code> .
Graceful shutdown	Pendente	Nao implementado.
Testes	Pendente	Sem arquivos de teste.
QA end-to-end	Pendente	Sem cenarios automatizados ou evidencias.
SaaS multi-tenant	Pendente	Roadmap futuro, nao implementado.

7 Comparativo por area funcional

7.1 Arquitetura principal

O planejamento previa uma aplicacao em camadas com FSM, debounce, PromptBuilder e ConfigLoader. Isso esta implementado. O codigo separa controllers, services, flow, types e utils, o que e positivo para manutencao.

O ponto de atencao e que a serializacao por usuario ainda e limitada ao debounce. Se uma conversa gerar processamento longo e o usuario mandar outra rajada, pode haver concorrencia entre processamentos do mesmo numero. Para producao, uma fila por usuario seria mais segura.

7.2 Inteligencia artificial

O projeto usa Gemini em duas funcoes: resposta principal e extracao de perfil. A resposta principal recebe prompt dinamico por estado e pode chamar ferramentas. A extracao tenta preencher nome, idade, objetivo e alvo da matricula.

O desenho e bom, mas regras criticas ainda estao concentradas no prompt. Em sistemas com pagamento, agenda e LGPD, o backend deve validar regras essenciais antes de executar a acao.

7.3 Funil comercial

O funil existe e cobre saudacao, qualificacao, apresentacao, objecoes, fechamento e transferencia humana. A maior divergencia e que a ordem atual de qualificacao coleta nome, alvo da matricula, idade e objetivo; o planejamento falava em objetivo e idade primeiro.

Tambem existe diferenca na regra teen: o documento fala 14–16 em alguns trechos, enquanto o prompt usa 12–17 em outros. Essa regra precisa ser unificada antes da entrega.

7.4 Asaas

O Asaas esta bem encaminhado para criacao de cliente, cobranca, assinatura, link e cancelamento. O maior problema e o webhook: hoje qualquer origem poderia simular pagamento confirmado se souber o endpoint e o formato do payload.

Esse e o principal item de seguranca antes de producao.

7.5 Google Calendar

As quatro operacoes principais existem. O sistema verifica disponibilidade antes de criar evento e suporta recorrência semanal.

Problemas atuais:

- Calendar ID hardcoded.
- Credencial esperada em arquivo local nao documentado adequadamente.
- Regras de horario comercial ficam no prompt, nao no backend.

7.6 ResponDi

O ResponDi esta implementado de forma funcional. O controller valida token, extrai respostas por titulo, normaliza telefone e atualiza/cria usuario.

O principal ajuste e reduzir logs com dados pessoais. Payload bruto com CPF, email e endereco nao deve ir para log de producao.

7.7 LGPD

Ha uma preocupacao real com LGPD: consentimento, exclusao por solicitacao, cascade delete e retencao de leads inativos.

Ainda faltam:

- Fluxo deterministico para acesso/correcao de dados.
- Politica de log sem dados pessoais.
- Retencao clara para clientes convertidos.

8 O que ja foi feito

- Servidor Express com rotas principais.
- Webhook WhatsApp com ACK imediato.
- Debounce por usuario de 5 segundos.
- Persistencia de usuario e historico.
- Normalizacao de telefone brasileiro.
- FSM conversacional.
- Extracao de perfil com Gemini.
- Resposta principal com Gemini.
- Prompt dinamico por estado.
- Tool calling com Calendar e Asaas.

- Envio de WhatsApp via Meta Graph API.
- Intake ResponDi.
- Cadastro, assinatura, cobranca e cancelamento no Asaas.
- Confirmacao de pagamento via webhook.
- Integracao Google Calendar.
- Exclusao LGPD.
- Retencao de leads inativos por 30 dias.
- Configuracao por JSON.

9 O que falta fazer

9.1 Prioridade critica

1. Validar webhook Asaas.
2. Remover instancias diretas de `new PrismaClient()`.
3. Adicionar graceful shutdown.
4. Corrigir busca das ultimas 20 mensagens.
5. Criar testes unitarios para regras centrais.
6. Criar `.env.example`.
7. Criar migrations Prisma versionadas.

9.2 Prioridade alta

1. Mover Calendar ID para variavel de ambiente.
2. Validar horario comercial no backend.
3. Adicionar idempotencia no webhook Asaas.
4. Remover logs de payload bruto com dados pessoais.
5. Atualizar README.
6. Criar checklist QA end-to-end.
7. Criar script/config PM2.

9.3 Prioridade media

1. Transformar regras criticas de prompt em validacoes backend.
2. Adicionar status de matricula/pagamento no banco.
3. Melhorar roteamento deterministico para PRM.
4. Tipar melhor sessoes e perfis.
5. Remover codigo morto.
6. Criar logs estruturados.

10 Plano de entrega recomendado

Semana	Foco	Entregas
1	Organizacao e riscos	README, envs, Prisma Singleton, historico correto, plano QA.
2	Seguranca	Webhook Asaas protegido, logs sensiveis removidos, Calendar env.
3	Testes essenciais	FSM, normalizador, PromptBuilder, calculos Asaas, ResponDi.
4	Funil e integracoes	Ajuste teen/PRM, midias nao suportadas, QA sandbox Calendar/Asaas.
5	Deploy e transferencia	Build limpo, PM2, runbook, checklist final, treinamento e aceite.

11 Critérios de aceite

- Build TypeScript limpo.
- Migrations aplicadas por fluxo controlado.
- Webhook Asaas protegido e idempotente.
- Logs sem dados pessoais sensiveis.
- QA manual documentado.
- Testes unitarios cobrindo regras centrais.
- Graceful shutdown.
- Configuracao de producao sem Calendar ID hardcoded.
- README/runbook coerente com o codigo real.
- Fluxos principais validados em sandbox.

12 Conclusao

O Artemis Bot tem uma base valiosa e funcional. Ele cobre grande parte do planejamento original e mostra que o produto pode virar uma automacao comercial real para a Confluence. Ao mesmo tempo, ainda nao deve ser vendido como producao madura sem a etapa de endurecimento tecnico.

A proposta correta e posicionar a entrega como conclusao profissional de um sistema ja iniciado: corrigir riscos, proteger integracoes financeiras, documentar operacao, testar fluxos e preparar deploy. Com esse enquadramento, a proposta recomendada de R\$ 38.000 por aproximadamente 5 semanas e defensavel, desde que o escopo seja fechado e os custos de terceiros fiquem fora do pacote.